**New York University — Tandon School of Engineering**
Department of Technology Management and Innovation

**IE-GY 9113B — Systems Integration: From ERP to Agentic AI**

# Study Note — Lab 2: Automate the Repeatable Part

Module 2: The Automation Ladder | Pre-work assigned Week 5 · Lab session Week 6 (Friday, October 9, 2026)

| For this lab week | |
|---|---|
| Assigned (lab week) | The module-by-module Make build guide in this note. Set up a free Make account and connect it to your Lab 1 sandbox before you start. |
| Pre-work (gated) | Assigned in Week 5, due before this session: BUILD your first-pass Make automation at home, following §3. Floor for teams that get stuck: arrive with the candidate, its trigger, rule, and field mappings sketched — and your specific error. The in-class session assumes you arrive with a built (or attempted) scenario. Also arrive with your BOARD DECK current through S5 — the slides you have already produced (S3 Context, S4 Strategic problem, S5 Backbone diagnosis from Weeks 1–2 and Lab 1). You fill S6–S7 in class from what you build today; you should not be catching up old slides during the lab. |
| Recall | Lab 1 (your stable-rules decision point = the RPA candidate; your Gemini-built sandbox; the rejection-path and legacy-mismatch rows you seeded), Week 4 (the three-test RPA-candidate filter), Week 5 (Center of Excellence, human-in-the-loop oversight). |
| Feeds | Board Memo §4 (Automation Journey) — completes Milestone 1. Deck slides S6–S7. |

## PART ONE · STUDENT GUIDE

## 1. Why This Lab, and What Changed Since Lab 1

Module 1 built the backbone and named the leverage gap. Module 2 begins to close it, and Lab 2 is where you stop diagnosing and start **building**. You take the one step you marked in Lab 1 as the stable-rules decision point — your RPA candidate — and you automate it: a working, multi-step automation that connects at least two systems and acts on your own diagnosed process. Then you break it on purpose. Finding where it breaks is the governance lesson, and it is the whole point of the lab — not a working demo, but an honest map of where a rules-based bot needs a human.

This lab completes **Milestone 1** and carries you from the backbone assessment (Board Memo §3) into the automation journey (§4). It builds directly on Lab 1 — this is not a fresh scenario. The step you automate is the step you already mapped; the data it runs

on is the sandbox you already built; the failure you provoke is the flaw you already seeded.

**One change from Lab 1 — read this.** In Lab 1, the build happened in class and time was tight; teams that started at home did best, and presentations got squeezed. Lab 2 fixes that: **you build the first pass at home**, and class is for refining, breaking, hardening, and presenting — unrushed. The module-by-module guide in §3 is written so you can build without an instructor in the room. Arriving with a built (or honestly attempted) scenario is the gate.

**On tools — outcome-first, stack-flexible**

The **Default stack** is **Make** (free tier: ~1,000 credits/month, 2 active scenarios, 3,000+ connectors, no code — *verify current limits*), reading System 1 = your Lab 1 sandbox (Google Sheets or Airtable) and writing to System 2 = a second free service the hand-off needs (Gmail, Slack, or a second sheet). A 3–4 module scenario costs 3–4 credits per run — well inside the cap. **AWS path**: Step Functions + EventBridge reach the same deliverable; a custom stack of your own is fine by approval. The graded standard is identical across all three — the automation and the governance thinking, never the platform or its complexity.

## 2. How This Lab Works — Build at Home, Refine in Class

Lab 2 is a **two-part deliverable**. The first build happens *before* class so class time goes to the parts that benefit most from the room: breaking the automation, deciding where oversight belongs, and presenting to peers who diagnosed different companies.

| Phase | When | What you do | Deliverable |
|---|---|---|---|
| Pre-work (gated) | Assigned Wk 5, due before Wk 6 | Build your first-pass Make scenario at home following §3. Run it once on your sandbox. | A built (or attempted) scenario. Floor: candidate + trigger/rule/mapping sketched + your specific error. |
| In-lab | Wk 6 session | Refine the scenario; break it on your Lab 1 seeded flaws; name the oversight the break implies; present. | Screenshot of the working workflow + a one-page post-mortem (what broke, oversight, ROI). |

| Phase | When | What you do | Deliverable |
|---|---|---|---|

**The gate — and the safety net.** Arrive with a built attempt. **If you got stuck at home**, that is expected, not a failure — bring the *specific error* you hit and the candidate/trigger/rule sketched; the first ~25 minutes of class is a recovery window with a minimum-viable path (§ Troubleshooting). What you cannot do is arrive with **nothing** — a refine-and-present session has nothing to refine if you never attempted the build.

**Two things are checked at the door:** (1) your built (or attempted) Make scenario, and (2) your board deck current through S5. Keeping the deck current as you build is the point of the course — each lab you add its slides and practise the story, so by December the deck assembled itself. Arriving with old slides unfilled means you spend class catching up instead of adding this lab's work.

**Two Make free-tier constraints to design around**

- **The free-tier scheduled trigger runs at most every 15 minutes.** Don't wait on it — use **Run once** to test, break, and re-run your scenario live.
- **Set up the Connection first.** Before building any module, create the Make↔Google (or Make↔Airtable) Connection and authorize it. Half of all first-build snags are here.

## 3. Before the Lab — Build Your First-Pass Automation (at home)

This is the load-bearing section: a module-by-module blueprint you can follow with no programming background. Budget ~1.5–2 hours. The shape is always the same — **trigger** → **condition** → **action** — connecting at least two systems. Substitute your own company's columns and rule; the structure holds.

### 3.1 Setup checklist (do this first)

1. **Create a free Make account** make.com. Verify you can create a new scenario.
2. **Have your Lab 1 sandbox ready.** If it's in Airtable, note the base and table. If it's in Excel, import it to Google Sheets (Make reads Sheets, not .xlsx). Confirm the tab with your process data and the *Status* column (where you seeded the rejection/rework path).
3. **Create the Connection.** In Make: Connections → add Google Sheets (or Airtable) → authorize with the account that owns your sandbox.

4. **Pick your second system.** The hand-off needs a destination: Gmail (an approval/notification email), Slack, or a second sheet/tab. Connect it too.

> **✓ Success looks like**
> A new blank scenario open in Make, with two Connections authorized (your sandbox + your second system). You haven't built anything yet — you're ready to.

## 3.2 The scenario blueprint — module by module

> **Build to the outcome, not the clicks — and use Maia if you're stuck**
> The module names below are one worked path; Make's interface changes and there is more than one right way. What matters is the **outcome each module produces**, not the exact clicks. Two things make this adaptive. **(1) If you don't know how to build a module, use Make's "Build with Maia"** — describe in plain English what you want (Citi Bike example: "when available bikes is below 20% of capacity, add a row to my Tasks sheet with the station ID and count") and let it wire the pieces. Then **check what Maia built against the rule you actually intended** — it will make reasonable guesses (a looser threshold, an extra label) you must verify and tighten. Getting Maia to build it, then correcting its rule, is itself the lab's skill. **(2) System 2 is swappable.** The hand-off can be a second sheet/tab (no new authentication), an email, or a Slack message. If a connection won't authenticate, **fall back to writing a second sheet** — the lesson is the hand-off, not the channel. Don't let an auth snag block the build.

Build the RPA candidate you tagged in Lab 1. The worked shape below uses an *order/credit-check* style step; map each module to your own candidate. Add modules left to right on the Make canvas.

**Module 1 — the Trigger (read System 1)**

1. **Add your sandbox as the first module.** Google Sheets → **"Watch New Rows"** (or, to test freely, **"Search Rows"**). Airtable → **"Watch Records."** Point it at your process tab. Set it to return the rows the automation should act on.

Config: choose the spreadsheet/base, the sheet/table, and (for Watch New Rows) the row where watching starts. For testing, **Search Rows** with a filter is easier because you control exactly which rows come through.

> **✓ Success looks like**
> Run once → the module returns real rows from your sandbox (you can see the field values — CustomerID, amount, status, etc.). If it returns nothing, widen the filter or check the Connection.

**Module 2 — the Condition (encode the rule = your decision point)**

2. **Add a Router or a Filter.** This module IS your stable-rules decision point, now in software. Set the exact logic your Lab 1 rule uses. Examples: **amount ≥ threshold**; or **status = "needs approval"**; or **available < reorder point**. A **Router** gives you two branches (e.g., auto-approve vs. route-for-review); a **Filter** simply passes or stops.

> **Tip**
> Keep the rule *complete and explicit* — every case decided, no "it depends." If you find yourself wanting a human to judge, you've wandered onto the judgment decision point, which is Lab 3's target, not this one.

> **✓ Success looks like**
> The Router/Filter shows your rows splitting the right way on a Run once — the ones that meet the rule go down one branch, the rest down the other (or stop).

**Module 3 — the Action (write to System 2)**

3. **Add your second system as the output.** On the "triggered" branch: Gmail → **"Send an Email"** (an approval/notification), or Slack → **"Create a Message,"** or Google Sheets → **"Add a Row"** to an *AutoActions* tab. Map the fields: which sandbox columns fill which parts of the message/row (e.g., CustomerID → subject, amount → body).

> **✓ Sanity-check your output before you call it done**
> State your rule explicitly and in one direction — e.g., "flag a station when available_bikes < 20% of capacity" (Citi Bike example). Then read the result with common sense: **you should get a handful of flagged rows, not most of your rows.** If nearly everything is flagged, the threshold is too loose — tighten it. If a station that is 90% full is flagged "empty," the rule has the direction backwards. If the same station appears several times, the scenario ran more than once (each run appends) — clear the output and Run once. Catching these is the point: a bot applies its rule with perfect confidence even when the rule is wrong, so validating the output against sense is part of the build.

> **✓ Success looks like**
> Run once → an email/Slack message/row actually appears, populated with real values from your sandbox. That is a working multi-system automation. **This is your 'first pass' — stop here for the gate.**

> **⚠ Verify live (free-tier specifics change)**
> Make's exact module names (Watch New Rows / Search Rows / Add a Row / Send an Email), the free-tier operations-per-run and monthly-credit caps, and the connector auth flow can change. If you hit an operations limit, narrow the trigger to a handful of rows — you're proving the mechanic, not processing the whole sandbox.

**What you bring to class.** A scenario of **3–4 modules** — trigger → condition → action — that runs once and produces a real output in a second system. If you got that far, you're ready to refine and break it. If you got stuck, bring the module where it stalled and the exact error message.

## 4. In the Lab — Refine, Break, Harden

You arrive with a built (or attempted) scenario. Class is where it earns the lab's real lesson.

### 4.1 Step 1 — Refine (get the happy path clean)

Run your scenario on your sandbox and confirm the routine cases flow correctly — the right rows trigger, the rule splits them properly, the output lands in System 2. Fix any field mappings that are off. Teams that arrived stuck: this is where the recovery window gets you to a working first pass (§ Troubleshooting).

### 4.2 Step 2 — Break it (the heart of the lab)

Now feed the automation the flaws you seeded in Lab 1. This is not a tangent — Lab 1 diagnosed and planted the bad data precisely so Lab 2 could trip on it. Your job is not to make it work perfectly; **your job is to find where it breaks.**

| Feed it this (from your Lab 1 sandbox) | What should happen | What it reveals |
|---|---|---|
| The rejection/rework rows — records you marked Status = 'Rejected' / 'Rework' | The rule mis-routes them, or silently skips them, or the output fires anyway on stale data. | The bot follows its rule with perfect confidence even when the case is wrong for it — no judgment, no doubt. |
| The legacy-mismatch rows — the non-reconciling source you seeded (e.g., LegacyStock disagreeing with live) | The automation acts on the wrong number: it triggers when it shouldn't, or misses a case it should catch. | A surface-level bot inherits bad data; it papers over a missing integration, it doesn't fix one (Week 4). |

| Feed it this (from your Lab 1 sandbox) | What should happen | What it reveals |
|---|---|---|
| A malformed / missing field — a blank where the rule expects a value | An error, or a silent pass with an empty output. | Where input validation and a human checkpoint belong. |

**Read the break as a place, not a bug.** Every failure you provoke marks a spot where a rule ran out and a human should be. Document each one carefully: **what you fed it, what it did, and what a person would have caught.** That documentation — not a flawless demo — is the deliverable.

## 4.3 Step 3 — Harden (name the oversight)

For each break, state the **human oversight** you would build in — the specific checkpoint that would have caught it. This is the bridge from rule-based automation to governance (Board Memo §5), and it is Week 5's human-in-the-loop question made concrete:

1. **A confidence / validation gate** — e.g., "if a required field is blank or out of range, stop and flag rather than send."
2. **An exception route** — e.g., "route rejected/rework rows to a human queue, not the auto-approve branch."
3. **A freshness / reconciliation check** — e.g., "confirm the source is the live feed, not the nightly legacy one, before acting."

You do not have to build the guardrail in Make — naming it precisely is the deliverable. "Rules-based" never means "no human needed": a bot follows a wrong rule confidently, so route edge cases, high-value transactions, and irreversible actions to a human on purpose.

## 4.4 Step 4 — The simple ROI (method, not a number we give you)

Your board memo lives on an honest estimate, so build one from **your own** scenario — we invent no figures for you. The method: take the step's *volume* (how many times a day/month), the *time per instance* a human spends, and the *labour rate*; that is the recurring cost the automation removes. Against it, put the one-time build effort plus any licence, and leave the **exception volume in** (the cases that still go to a human). Express it as a rough payback period. An honest estimate with the exceptions costed beats an optimistic one that wishes them away.

> **Tip**
> Keep it order-of-magnitude and defensible. A board funds a *thread outcome* — cash conversion recovered, customer wait removed — tagged to a number, not a precise-looking figure you can't defend.

### 4.5 Step 5 — Present (8 minutes) + what to show

You present from your **board deck** — the real capstone template, not a throwaway lab slide. You walked in with it current through **S5** (S3 Context, S4 Strategic problem, S5 Backbone diagnosis — from Weeks 1–2 and Lab 1). During this lab you fill **S6 (Automation opportunity) and S7 (Automation journey)** from the automation you build and break — so by the time you present, the deck tells the story start to where you got. You have **8 minutes** + **4 minutes** of feedback, and the time is deliberately uneven: **go quickly over the older slides (S3–S5) and spend most of the time on what you added today (S6–S7).** We want depth on the new work, not a tour of old slides. This is the same deck you defend in December.

| Minutes | Deck slide | Show this |
|---|---|---|
| **~2 min** | **S3–S5 — story so far** | Walk it at pace: company & strategic problem (S3–S4), and your Lab 1 diagnosis — friction chain + stable-rules candidate (S5). A sentence or two each; context, not the point. |
| **~2 min** | **S6 — the automation** | What your Make scenario connects and does on the happy path — a quick demo or screenshot. |
| **~2.5 min** | **S7 — where it broke** | The input you fed it, the failure it produced, and the **human oversight** the break implies. The heart — spend the most time here. |
| **~1.5 min** | **S7 — ROI & thread** | Your ROI method and the rough prize, tagged to a thread (Cash / Customer / Operations). |

**Submit end of day.** A **screenshot of the working workflow** + a **one-page post-mortem**: what you built, where it broke, the human oversight you would add, and your simple ROI estimate. This is the 'Lab 2 automation scenario' in **Board Memo §4** — point your memo at it.

## 5. Troubleshooting & the No-Build Fallback

Capturing where it breaks is the lesson — so known failure modes are content, not embarrassment. If you got stuck at home, start here; the first ~25 minutes of class is a recovery window.

### 5.1 Common first-build snags

| Symptom | Likely cause / fix |
|---|---|
| Trigger returns no rows | Connection points at the wrong sheet/table, or the filter is too narrow. Use Search Rows with a simple filter to test. |
| “Connection” error / auth failed | Re-authorize the Connection with the account that owns the sandbox; Make→Google and Make→Airtable are separate Connections. |
| Scenario won’t save / trigger greyed out | On the free tier, use Run once for testing rather than scheduling; the 15-minute minimum interval doesn’t apply to manual runs. |
| Output fires but fields are empty | Field mapping missing — open the action module and map each field to a value from the trigger module explicitly. |
| Ran out of operations/credits | Narrow the trigger to a handful of rows; a 3–4 module test run costs 3–4 credits. You’re proving the mechanic, not processing everything. |

### 5.2 The no-build fallback (in class, ~25 min)

**Minimum-viable scenario — if you arrived with nothing running.** Build the smallest possible version: (1) **Search Rows** on your sandbox, filtered to ~5 rows; (2) a **Filter** with one simple rule; (3) one **Add a Row / Send Email** action. Run once. That is a legitimate first pass — then break it on one seeded flaw and name one oversight. A minimal automation broken honestly meets the deliverable; do not spend the class polishing a bigger one you can’t finish.

## Where This Lab Sits — and What It Hands Forward

Lab 2 completes **Milestone 1** and turns the diagnosis into evidence. It **recalls** from Lab 1 the stable-rules RPA candidate, the sandbox, and the seeded flaws; from Week 4 the three-test candidate filter; from Week 5 the Center-of-Excellence and human-oversight language. It **applies** them: a working automation on your own process, broken on your own data, with the oversight named.

It **hands forward** two things. To your **board memo**: the automation scenario is §4 (Automation Journey), and on the deck it fills S6–S7 — completing the transition from the backbone assessment (§3) to the automation journey (§4). To **Lab 3**: the break you just documented is the preview of why rules fail at judgment. The edge case a human would have caught intuitively is exactly the **judgment decision point** you also marked in Lab 1 — the step an *AI agent* takes in Lab 3. You do not build agents here; you feel, first-hand, why they’re needed.

## Your Study Questions

Bring reasoned answers; they are designed to be argued, not recited.

1. **Where did your automation break, and why is that break a governance finding, not a bug?** Name the input, the failure, and the human oversight it implies.
2. **Your automation broke on an edge case a human would have caught intuitively.** What does that tell you about the limits of rules-based systems — and about what we risk when we remove the human?
3. **State your ROI method for this scenario.** What did you leave in (exception volume, oversight cost) that an optimistic estimate would drop, and why does leaving it in make the case stronger?
4. **Which of your breaks points at the judgment step you marked in Lab 1?** Why can't a rule fix it — and what would an agent need to that a bot lacks?

## Running Glossary

**Trigger → condition → action.** The shape of a Make scenario: a trigger reads data from System 1, a condition (Router/Filter) encodes the rule, an action writes to System 2. The rules-based automation pattern this lab builds.

**Scenario (Make).** A Make automation: the connected set of modules (trigger, condition, actions) that runs as one workflow.

**Connection (Make).** The authorized link between Make and an external system (your sandbox, Gmail, Slack). Set up before building; a frequent first-build snag.

**Run once.** Make's manual single execution — used to test and break a scenario without waiting on the free tier's 15-minute scheduled interval.

**Break-it / failure mode.** Deliberately feeding an automation an edge case (a seeded flaw) to find where it fails — which is where human oversight belongs. The core deliverable of Lab 2.

**Human-in-the-loop oversight.** A deliberate human checkpoint on an otherwise-automated flow — a validation gate, an exception route, a freshness check. Named here; governed in Week 5; carried to Board Memo §5. (Introduced Week 5.)

**RPA candidate.** A step that is repetitive, rules-based, and stable — the stable-rules decision point from Lab 1, and this lab's automation target. (Introduced Week 4.)

**Post-mortem.** The one-page write-up: what you built, where it broke, the oversight you would add, and a simple ROI estimate. Feeds Board Memo §4.